

Proyecto 2 - Freesound Audio Tagging

Taller de Aprendizaje Automático

Luciana García

Santiago Márquez

Santiago Rodríguez

5 de julio de 2026

1. Introducción

El problema consiste en asignar etiquetas sonoras a audios de Freesound. Es una tarea *multi-label*: una grabación puede contener varias fuentes de sonido y la salida debe ser una probabilidad para cada una de las 80 clases. La métrica usada por Kaggle es *label-weighted label-ranking average precision* (lwlrap), que premia que las etiquetas correctas queden bien rankeadas para cada audio.

La primera entrega consolidada usaba un ensamble de tres ramas entrenadas 100 épocas sobre `train_curated`. Ese sistema obtuvo **0.67126** en el private leaderboard de Freesound Audio Tagging 2019. La versión final de este informe toma ese modelo como punto de partida y agrega una segunda etapa: continuar el entrenamiento con datos curados y ruidosos, pero controlando el impacto de `train_noisy`. El resultado final fue **0.68122** en el private leaderboard de Freesound 2019 y **0.64307** en el leaderboard público del Kaggle del curso.

2. Datos y métrica

El dataset contiene tres fuentes principales: `train_curated.csv`, `train_noisy.csv` y `sample_submission.csv`. El conjunto curado es pequeño pero confiable; el conjunto ruidoso tiene más ejemplos, aunque sus etiquetas son menos seguras. Esa diferencia motivó no concatenar ambos conjuntos sin control.

Split	Filas	Rol	Uso final
train_curated	4970	etiquetas confiables	base y 50 % de cada batch
train_noisy	19815	mayor cobertura, etiquetas ruidosas	50 % de cada batch, peso 0.30
test	3361	audios a predecir	submission final

Cuadro 1: Conjuntos usados. El fine-tuning usa noisy con menor peso para aportar variabilidad sin dominar la señal curada.

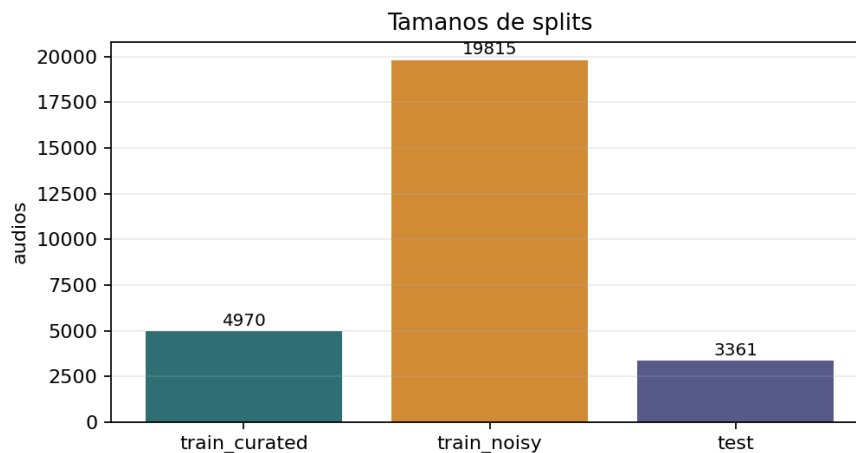


Figura 1: Tamaño relativo de los tres splits.

La métrica lwlap se puede leer como un promedio ponderado de precisión en ranking. Para cada audio se ordenan las clases por probabilidad y se mira si las clases verdaderas aparecen arriba:

$$\text{lwlap} = \sum_{c=1}^C w_c \frac{1}{N_c} \sum_{i: y_{ic}=1} \text{precision}_{i,c}.$$

Por esto el promedio de modelos es natural: ramas con errores distintos pueden mejorar el ranking final aunque cada una no sea perfecta.

3. Representación de audio

Cada archivo .wav se convierte en una imagen log-mel:

`.wav ->mono ->44.1 kHz ->MelSpectrogram ->log/dB ->crop/pad ->normalización`

Se usan 128 bandas Mel. El eje horizontal representa tiempo y el eje vertical frecuencia perceptual. Las ramas principales usan entradas 128 x 512; la rama temporal larga usa 128 x 1024.

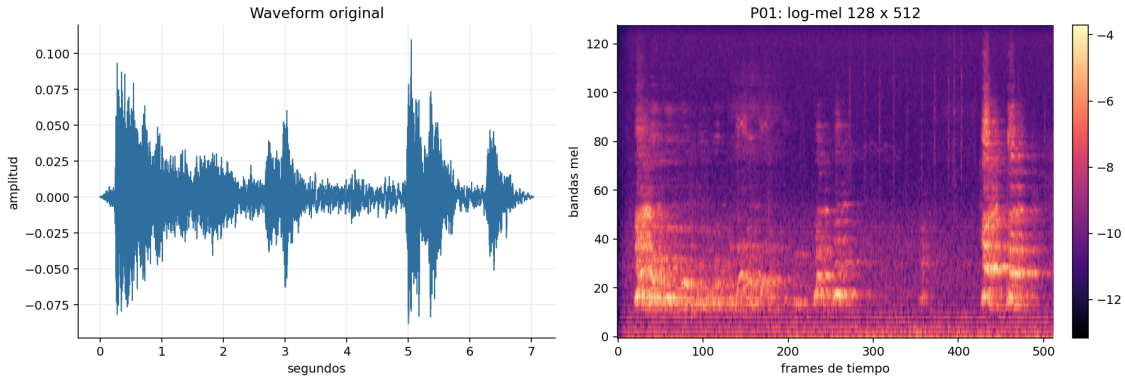


Figura 2: Pasaje de forma de onda a imagen log-mel. La red aprende patrones tiempo-frecuencia sobre esta representación.

4. Modelo base: ensamble 3-way e100

Antes del experimento final se fijó un ensamble defendible de tres ramas:

- **H100:** CNN separable-residual con cabeza densa, log-mel 128 x 512, normalización por clip.
- **G100:** CNN separable con cabeza temporal BiGRU y normalización global por banda Mel.
- **F100:** misma familia temporal, pero con ventana 128 x 1024 para conservar más contexto.

La combinación usada fue:

$$0,25 H100 + 0,375 G100 + 0,375 F100.$$

Este sistema obtuvo **0.67126**. También se habían probado variantes más complejas, por ejemplo bagging, con score **0.67674**; sin embargo, esa mejora era menos limpia conceptualmente. El experimento final buscó una mejora manteniendo la misma arquitectura de tres caminos.

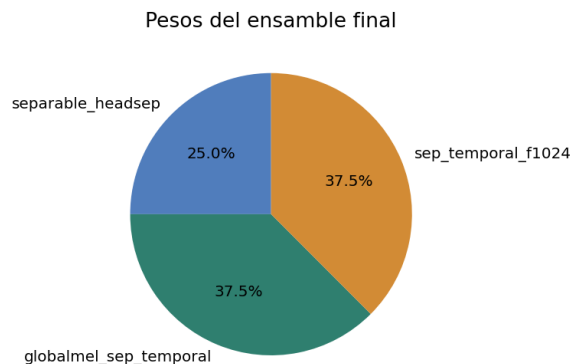


Figura 3: Pesos del ensemble final. Se conservaron los mismos pesos que en el modelo 3-way e100.

5. Experimento final: noisy fine-tune

5.1. Hipótesis

La hipótesis fue que el modelo e100 ya había aprendido una representación fuerte desde etiquetas curadas, pero aún podía beneficiarse de la variedad de `train_noisy` si la segunda etapa era conservadora. Por eso se aplicaron cuatro restricciones:

1. partir de los checkpoints finales del ensemble 3-way;
2. usar learning rate bajo, $1e-4$, con scheduler coseno;
3. formar batches mitad `curated` y mitad `noisy`;
4. pesar las filas noisy con 0.30 en la pérdida.

Además se usó data augmentation online sobre log-mel: time shift, time mask, frequency mask y ruido gaussiano.

5.2. Configuración

Rama	Frames	Peso ensemble	Segunda etapa
separable_headsep	512	0.250	30 épocas, batches 50/50, noisy 0.30
globalmel_sep_temporal	512	0.375	30 épocas, global-mel, noisy 0.30
sep_temporal_f1024	1024	0.375	30 épocas, memmap para evitar OOM, noisy 0.30

Cuadro 2: Configuración de la continuación final.

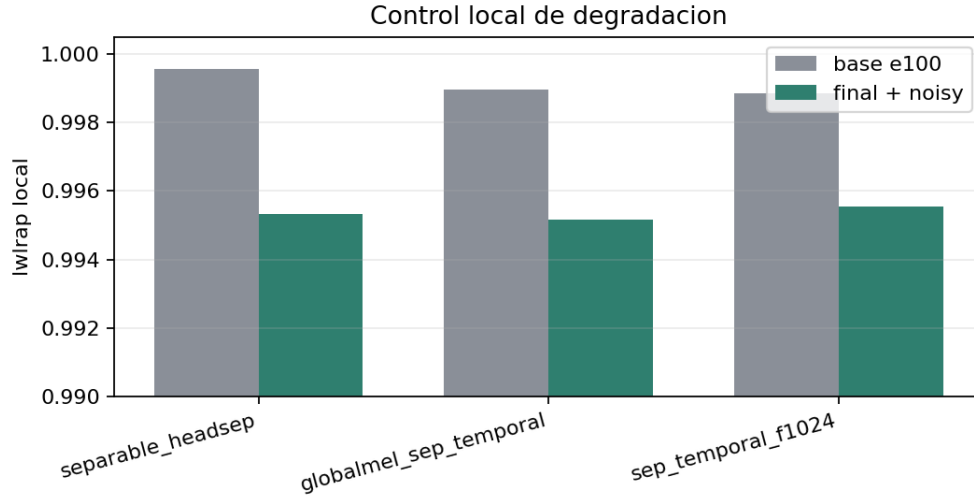


Figura 4: Control local de degradación. La validación local no es independiente: los checkpoints base fueron entrenados con todo curated.

La Figura anterior no se interpreta como estimación real del leaderboard, porque los modelos base ya habían visto todo `train_curated`. Sirve para detectar degradaciones grandes; la decisión final se toma con Kaggle.

6. Resultado del flujo final

El resultado que se entrega no corresponde al modelo 3-way e100 aislado, sino al flujo completo de dos etapas: primero se entrenan las tres ramas sobre `train_curated` durante 100 épocas y luego se continúa cada una con `train_curated + train_noisy` usando batches balanceados, menor learning rate, augmentations y peso reducido para las etiquetas ruidosas. La submission final se obtiene promediando las tres ramas fine-tuned con los mismos pesos del ensamble base:

$$0,25 H_{ft} + 0,375 G_{ft} + 0,375 F_{ft}.$$

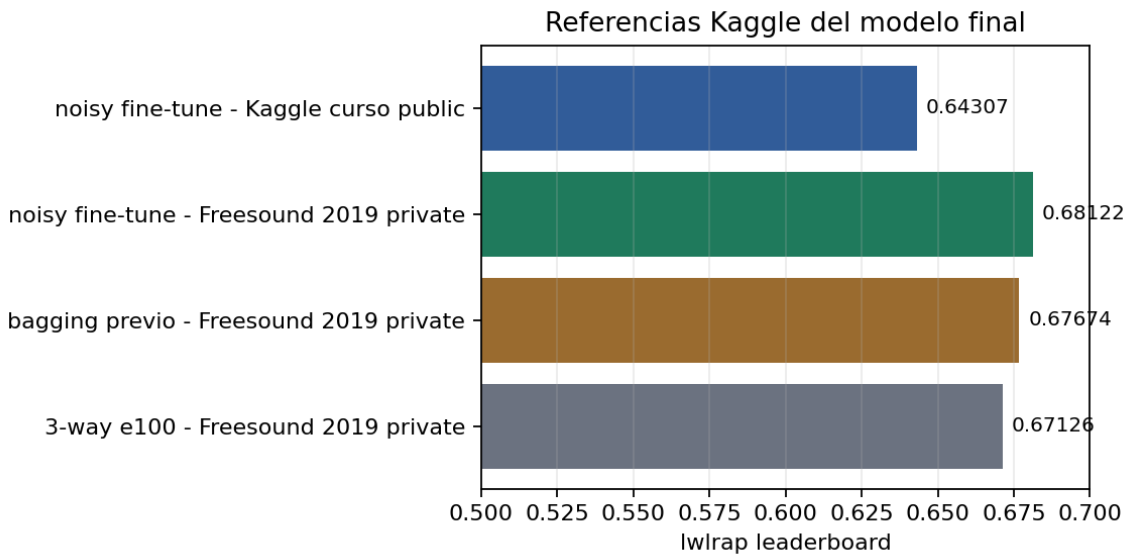


Figura 5: Scores relevantes del flujo final. El Kaggle del curso usa una variante del dataset y se reporta separado.

Sistema	Score	Lectura
3-way e100 sin noisy	0.67126	punto de partida
Mejor variante previa registrada	0.67674	bagging, más compleja
Flujo final completo, Freesound 2019 private	0.68122	modelo entregado
Flujo final completo, Kaggle del curso public	0.64307	referencia en variante de datos del curso

Cuadro 3: Resultados principales. El modelo entregado es la versión fine-tuned, no el checkpoint e100 previo.

La mejora contra el 3-way e100 sin la segunda etapa es:

$$0,68122 - 0,67126 = 0,00996.$$

Contra la mejor variante previa registrada:

$$0,68122 - 0,67674 = 0,00448.$$

El CSV final tiene 3361 filas y 80 columnas de etiquetas. Las probabilidades quedan en $[0, 1]$, el orden de `fname` coincide con `sample_submission.csv` y el hash SHA256 es:

`b29288caa6e7b37b29e830a29655decc7c6bc8110ca3a40b828f4dd2f5fabdccc`

Además se preparó el código de entrega como pipeline reproducible: el notebook parte de los datos crudos, reconstruye las representaciones log-mel, entrena las ramas base, ejecuta la continuación con noisy y vuelve a generar el ensamble. También se incluye una versión larga del notebook con el código fuente embebido, pensada para auditoría o defensa.

6.1. Lectura de predicciones del modelo final

Para inspeccionar cualitativamente el comportamiento del ensamble final se revisaron ejemplos del split local usado durante el fine-tuning con noisy. Esta lectura no se toma como validación independiente, porque los checkpoints base ya habían sido entrenados con todo `train_curated`; su valor es mostrar si el ranking de clases del modelo final resulta interpretable.

Audio	Etiquetas reales	Top-5 predicho	Lectura
<code>cce07823.wav</code>	Fill (with liquid), Gurgling, Sink, Water tap	Fill 0.993; Water tap 0.980; Sink 0.977; Gurgling 0.976; Trickle 0.968	las cuatro etiquetas reales quedan arriba; la clase extra también es de agua
<code>38b700ca.wav</code>	Applause, Cheering, Crowd	Applause 1.000; Crowd 0.999; Cheering 0.999; Harmonica 0.073; Run 0.036	ranking correcto y muy concentrado en el evento de público
<code>a61153d2.wav</code>	Acoustic guitar, Strum	Acoustic guitar 1.000; Strum 1.000; Bass guitar 0.020; Electric guitar 0.005; Gong 0.003	identifica instrumento y acción; los falsos positivos son instrumentos cercanos
<code>c197ef4f.wav</code>	Bicycle bell	Bicycle bell 1.000; Glockenspiel 0.180; Chink/clink 0.067; Microwave 0.029; Shatter 0.023	top-1 correcto; las alternativas son sonidos metálicos breves, acústicamente plausibles

Cuadro 4: Ejemplos de interpretación del ensamble final con noisy fine-tune.

7. Conclusión

La contribución final no fue cambiar la arquitectura, sino cambiar el protocolo de entrenamiento. Primero se aprende de etiquetas confiables durante 100 épocas; luego se usa el conjunto ruidoso como regularizador y fuente de cobertura, pero con batches balanceados, peso reducido y learning rate bajo. Esta estrategia mantiene la claridad del ensamble de tres caminos y mejora el resultado de leaderboard. Para la defensa, el mensaje central es que `train_noisy` no se descartó ni se incorporó ingenuamente: se incorporó con una política explícita de confianza parcial.